

Kurtosis

By Onkar_Raskar

FY B.Tech EC

VJTI ##

About me

I am Onkar Raskar from First year studying Electronics Engineering at VJTI Mumbai. I am interested in Competitive programming and I have a strong grasp of the basics of programming languages like C++ and python. I have also built small projects like snake game and simple ping pong game using python. I want to work in the field of Machine learning and understand it to great depths.

About Project

ABSTRACT

Modern industrial systems are increasingly required to operate with minimal downtime, yet they rely on mechanical components that inevitably degrade. Traditional maintenance strategies—either scheduled component replacements or reactive repairs—are highly inefficient and struggle to prevent cascading hardware failures. While advanced deep learning models can ingest raw sensor data to monitor machine health, they often act as black boxes, providing little engineering intuition about the physical root cause of a fault.

This project proposes an end-to-end Condition-Based Predictive Maintenance system built from first principles. By combining rigorous, physics-grounded signal processing with both explainable classical machine learning (Regularized SVMs and XGBoost) and advanced representational deep learning (Gramian Angular Fields and Unsupervised Anomaly Detection), this system will demonstrate how domain knowledge and end-to-end learning can be rigorously evaluated and combined. The project will bridge one-dimensional acoustic analysis with two-dimensional computer vision techniques to create a highly robust diagnostic pipeline capable of listening to machines, understanding their behavior, and predicting failures before they occur.

Problem Statement

Industrial rotating machines—motors, pumps, compressors, and turbines—power everything from manufacturing plants to transportation networks. These machines emit subtle vibration and acoustic signatures that change detectably weeks before a catastrophic failure event occurs. The cost of missing these signals and experiencing unplanned downtime is enormous: modern factories commonly lose ₹2–10 lakhs per hour, with a single bearing failure triggering cascading disruptions across an entire production line.

Current industrial practice relies heavily on two flawed methodologies: scheduled preventive maintenance (replacing parts every N hours regardless of their actual condition, leading to massive part waste) or reactive maintenance (fixing machinery only after it breaks, leading to halted production). Both are financially and operationally wasteful.

While Condition-Based Predictive Maintenance using Machine Learning offers a solution, engineers face an overfitting dilemma: models trained purely on raw data often learn the specific acoustic noise of a single factory floor rather than the underlying physics of a failing machine. There is a critical need for an intelligent system that not only detects operational anomalies with high accuracy but can mathematically isolate and explain the physical signal traits causing the failure, enabling just-in-time, targeted intervention.

Foundational Knowledge

Mechanics of Rotational Vibration & Statistical Textures

To build an intelligent diagnostic system, we must first understand the physical geometry and mechanics of the hardware. Industrial bearings consist of four primary components: inner races, outer races, rolling elements, and a cage holding the assembly together. When a localized defect develops on any of these surfaces due to wear or metal fatigue, the rolling elements strike the defect periodically during operation.

This interaction generates a repeating sequence of mechanical impacts. Because these impacts correspond directly to the physical rotation of the machine, they occur at specific, predictable frequencies that are mathematically derivable from the bearing's physical dimensions and the shaft's rotational speed. These characteristic fault frequencies—such as the Ball Pass Frequency Outer (BPFO) or Ball Pass Frequency Inner (BPFI)—act as the distinct, underlying fingerprints of mechanical degradation.

The core difficulty in predictive maintenance is that these characteristic fault frequencies are virtually invisible in a raw time-series accelerometer signal. In the time domain, the vibration waveform appears as chaotic, high-amplitude noise. To extract meaningful diagnostic data, the system must transform the 1D temporal signal into the frequency domain using a Fast Fourier Transform (FFT).

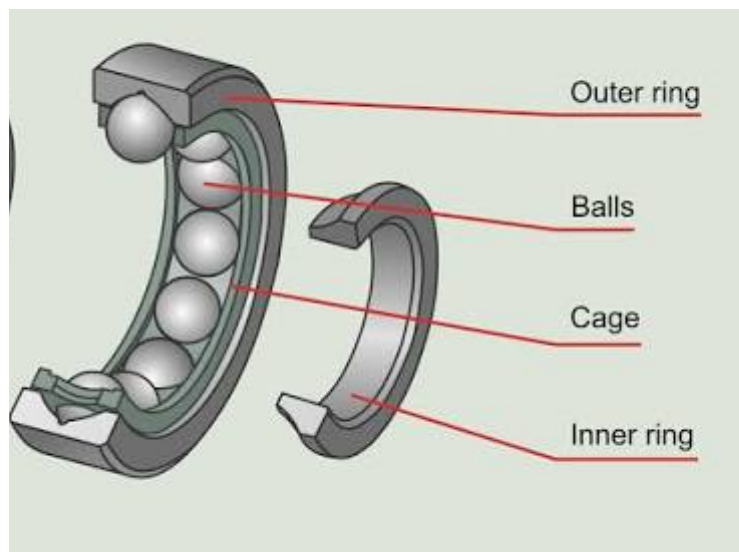
Once transformed, the hidden frequency patterns reveal sharp energy spikes. From this, we mathematically extract statistical health metrics to capture the "texture" of the signal:

Root Mean Square (RMS): Measures the overall vibrational energy and power of the signal.

$$X_{RMS} = \sqrt{\frac{1}{N} \sum_{i=1}^N x_i^2}$$

Kurtosis: A fourth-order statistical moment that measures the "tailedness" or impulsiveness of the signal. It is a highly sensitive indicator of the sharp, early-stage impacts of a localized bearing fault, often spiking long before RMS shows significant deviation.

Crest Factor: The ratio of the peak amplitude to the RMS value, indicating the severity and sharpness of the mechanical impacts.



Phase 1: Interpreting the Machine (Domain Knowledge Pipeline)

The main objective of Phase 1 is to answer an age-old question in modern AI: **Does domain knowledge beat end-to-end learning?** To figure this out, we will build two completely different pathways to analyze the machine's vibration data and compare which one performs better.

Pathway 1: The Domain Knowledge Approach

This approach relies on human engineering. Instead of feeding raw, messy noise to an AI, we first use signal processing (like a Fourier Transform) to clean the data and extract specific, physics-based clues. We will extract features like **Kurtosis** (which measures the sharpness of mechanical impacts) and specific **Fault Frequencies** (like BPFO, which tells us exactly if the outer ring of the bearing is damaged).

Once we have these refined clues, we will feed them into a series of classical machine learning models. To find the optimal diagnostic tool, we will trace the evolutionary "story" of decision-based AI, seeing how each model attempts to fix the flaws of the one before it:

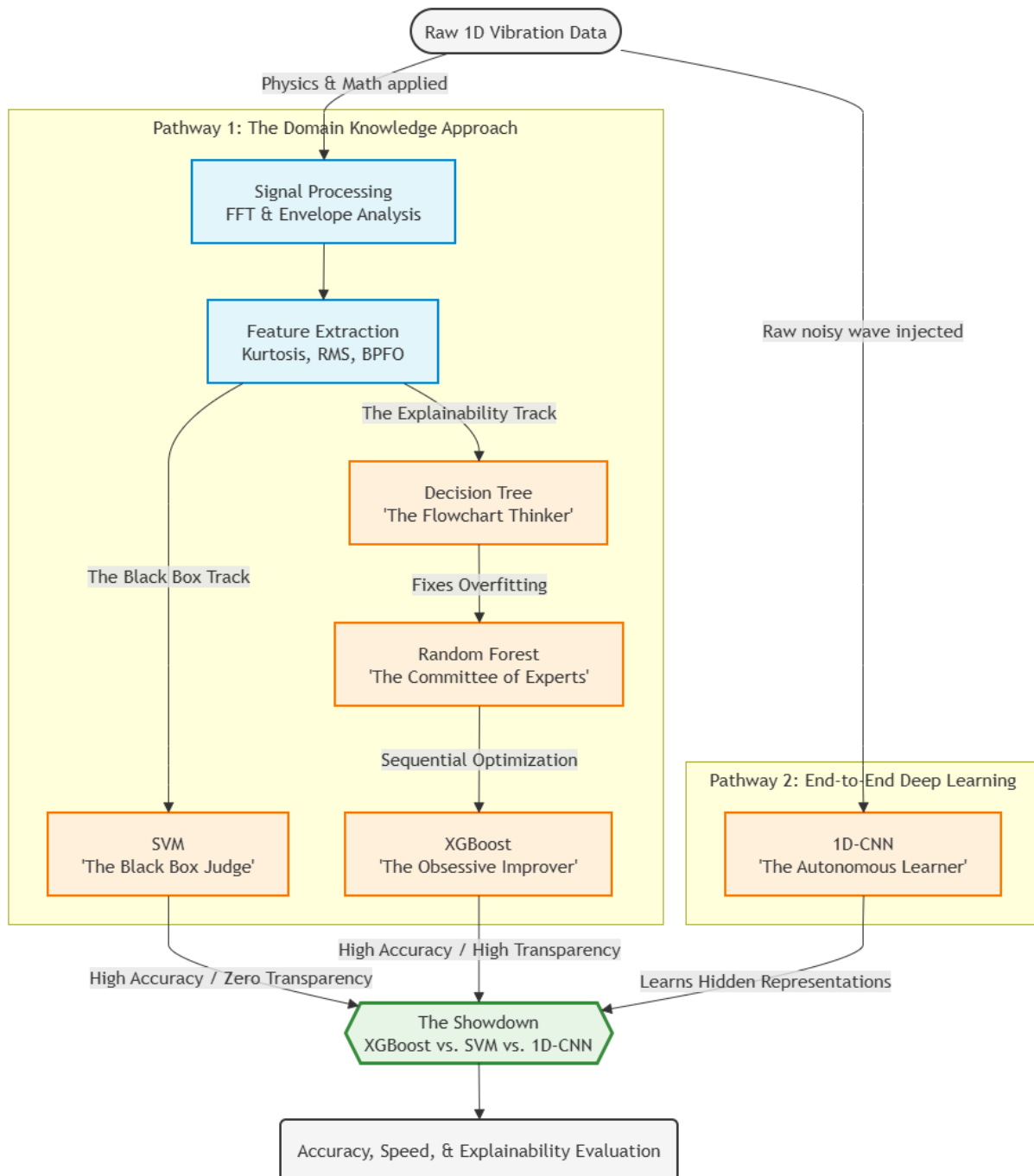
- **Model 1: The Decision Tree** This is the simplest form of artificial intelligence. It approaches the problem like a factory technician using a flowchart. It looks at the data and asks a series of simple Yes/No questions (e.g., *"Is the Kurtosis greater than 3.0? If yes, is the RMS high?"*). While it is highly interpretable and fast, its fatal flaw is that it tends to overthink. It often memorizes the specific background noise of our training data (a problem called overfitting) rather than learning the actual, universal rules of a broken machine.
- **Model 2: The Random Forest** To fix the Decision Tree's overfitting problem, we upgrade to a Random Forest. Instead of relying on one tree, this model builds a massive "forest" of hundreds of different decision trees. Each tree is only allowed to look at a random, limited subset of the data. When diagnosing a machine, all hundreds of trees cast a vote. By relying on this "wisdom of the crowd," the Random

Forest smooths out individual errors, creating a much more stable and reliable prediction than any single tree could achieve.

- **Model 3: Support Vector Machine** While the decision tree models attempt to reason through the problem, the Support Vector Machine (SVM) takes a completely different approach: it optimizes for pure mathematical accuracy at the cost of human readability. Imagine plotting our extracted machine data (like Kurtosis and RMS) on a 3D graph. Often, the "Healthy" and "Faulty" data points are tangled together. The SVM uses a technique called a Radial Basis Function (RBF) to project this data into a complex, higher-dimensional space, drawing intricate, invisible boundaries that perfectly fence off the failing machines.

However, its fatal flaw for industrial use is its total lack of transparency. It operates as a complete "Black Box." If a factory engineer asks the SVM *why* it flagged a machine, the model cannot point to a specific spike in Kurtosis or BPFO; its only reasoning is that the data fell outside its mathematical boundary. We include the SVM to represent the extreme end of the AI spectrum: a model with legendary accuracy that completely sacrifices the explainability required on a factory floor.

- **Model 4: XGBoost** This is the ultimate, cutting-edge evolution of the tree approach. Where a Random Forest builds its trees independently, XGBoost builds them *sequentially*. Every single time it generates a new tree, that tree is specifically designed to hunt down and fix the exact mistakes made by the previous tree. Furthermore, XGBoost acts as our Chief Explainer. It calculates a "Feature Importance Score," allowing it to tell engineers exactly *why* a machine is failing (e.g., *"I flagged this as an Outer Race fault because the BPFO frequency spiked"*). Recent literature proves this model achieves peak accuracy (outperforming other tree methods) while maintaining total transparency.



Pathway 2: The End-to-End Approach (Deep Learning)

This approach completely ignores human engineering. We bypass the math, skip the feature extraction, and feed the raw, noisy vibration wave directly into a deep learning model.

- **1D-CNN (Convolutional Neural Network):** This model acts as an autonomous learner. By processing the raw time-series wave through multiple hidden layers, the CNN attempts to figure out its own patterns and rules for what a failing machine sounds like, without any hints from us about Kurtosis or physics.

The Showdown

At the end of this phase, we will compare Pathway 1 against Pathway 2. We will look at their accuracy, speed, and reliability to finally conclude whether manually extracting features (like Kurtosis) genuinely helps the AI, or if a neural network learning from pure raw signals is the superior choice for predictive maintenance.

Phase 2: Descent to Madness (End-to-End Deep Learning)

While Phase 1 proves we understand the physics of a failing machine, Phase 2 addresses a massive real-world problem: industrial factories do not have perfectly balanced datasets. In reality, machines run perfectly 99% of the time. Factory engineers have terabytes of "healthy" data, but very few recordings of actual catastrophic failures.

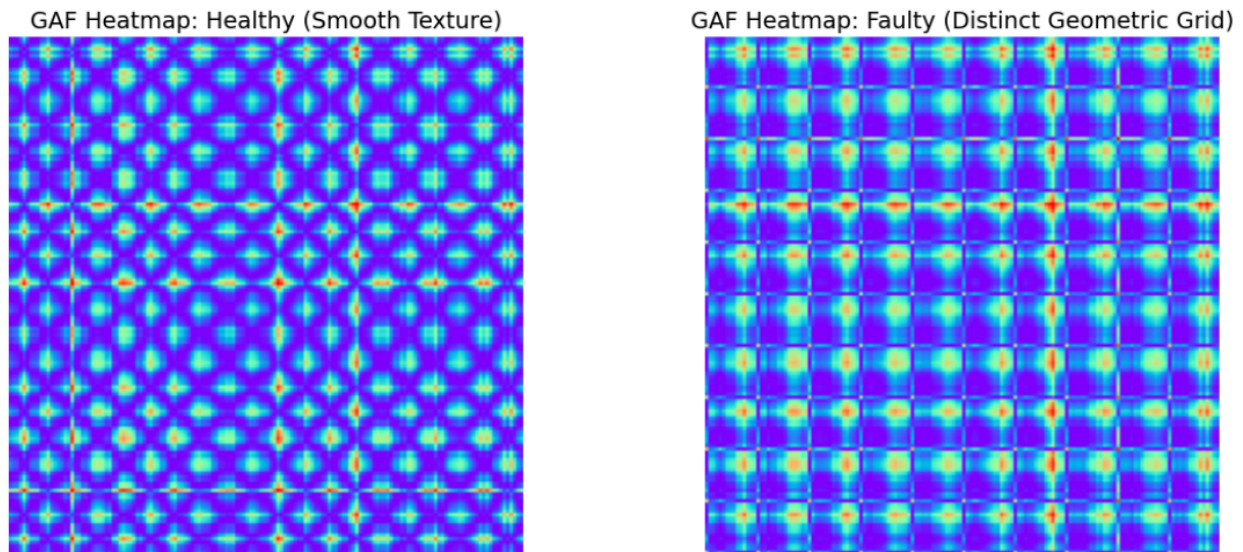
To solve this, Phase 2 abandons traditional signal processing and classification, shifting entirely to visual transformations and unsupervised learning.

Step 1: Transforming Time into Vision

A vibration signal is just a 1D squiggly line over time. While neural networks can process this, they are much better at analyzing pictures. We will use a mathematical technique called **Gramian Angular Fields (GAFs)** to wrap our 1D time-series signal into a 2D matrix.

Think of this as converting a sound wave into a highly detailed visual heatmap or QR code. The moment we convert the vibration into an image, a massive door unlocks: we can now use state-of-the-art Computer Vision

models (like ResNet or EfficientNet). These models were originally trained on millions of standard photos to detect edges, shapes, and textures. We will repurpose their powerful visual "eyes" to look for visual defects in our vibration heatmaps.



Step 2: Unsupervised Anomaly Detection

In Phase 1, we had to explicitly teach the AI, *"This is a healthy machine, and this is a broken machine."* Because real factories lack "broken" data, this approach does not scale.

Instead, we will pivot to **Anomaly Detection** using a generative AI architecture called **Normalizing Flows**. The logic here is entirely inverted. We will not teach the AI how a machine breaks. Instead, we will train the model *exclusively* on the vibration signatures of perfectly healthy bearings. The AI will deeply learn the exact mathematical "sound of health."

During real-time monitoring, the AI simply listens to the machine. If the vibration shifts and no longer matches the learned healthy profile, the system instantly flags it as an **anomaly**. This is a massive leap forward because it allows the AI to predict failures without ever needing to see a broken machine during its training.

The Final Validation

To prove this system actually works in the real world and hasn't just memorized the background noise of our specific test dataset, we will run a cross-validation test. We will take our AI—trained on the CWRU dataset—and force it to analyze a completely unfamiliar set of factory recordings (the MFPT dataset). If our visual transformations and anomaly thresholds hold up, we will have built an end-to-end predictive maintenance pipeline capable of universally diagnosing industrial failure.

Datasets and Data Format

To train and test our AI, we need high-quality recordings of machines in various states of health.

- **Primary Dataset (The Training Ground):** We will use the **CWRU (Case Western Reserve University) Bearing Dataset**, which is the gold standard for predictive maintenance research. The data consists of 1D accelerometer readings (recordings of physical vibrations) sampled at 12 kHz. It contains 10 different classes: one perfectly healthy baseline, and nine fault classes categorizing different types of damage (Inner Race, Outer Race, Ball Fault) at varying levels of severity.
- **Secondary Dataset (The Reality Check):** To ensure our model hasn't just memorized the specific background noise of the CWRU laboratory, we will test it against a completely unseen dataset: the **MFPT (Machinery Failure Prevention Technology)** dataset. If the AI performs well here, we know it has truly learned the physics of machinery.

Evaluation Metrics

To objectively measure the success of both Phase 1 and Phase 2, we will use the following metrics:

- **Classification Accuracy & F1-Score:** Measures how perfectly the Phase 1 models can separate healthy machines from failing ones,

while penalizing false alarms (False Positives) which cause unnecessary factory shutdowns.

- **Feature Importance Weight:** A percentage score generated by XGBoost that ranks exactly *which* physical metric (like Kurtosis or RMS) contributed the most to the AI's final decision.
- **Anomaly Detection Recall:** Used in Phase 2 to measure how successfully the generative model catches hidden failures before they happen. In industrial settings, a missed fault (False Negative) is catastrophic, so high recall is our top priority.
- **Generalization Penalty:** The drop in accuracy when the model trained on the CWRU dataset is tested against the completely unfamiliar MFPT dataset. A lower penalty means a more robust, real-world-ready AI.

Timeline

Week 1: The Data Science Foundation Dedicated focus on mastering the core Python data toolkit. Learning to utilize NumPy for high-speed matrix and array manipulation, Pandas for structuring and cleaning datasets, and Matplotlib/Seaborn for visualizing complex data trends.

Week 2: Machine Learning & Signal Processing Basics Shifting focus to classical AI and domain knowledge. Understanding the API architecture of Scikit-learn and studying how Decision Trees (XGBoost) operate. Exploring the basics of signal processing, specifically learning how Fast Fourier Transforms (FFT) extract frequencies from 1D waves.

Week 3: Deep Learning Frameworks Learning the fundamentals of deep learning using PyTorch (or TensorFlow). Understanding how to build basic Convolutional Neural Networks (CNNs), manage tensors, and utilize pre-trained models. Familiarizing myself with the CWRU dataset structure to prepare for the execution phase.

Week 4: Phase 1 (Data Prep & Classical Models) Applying the learned tools to the real dataset. Writing scripts to clean the data, execute the FFTs, and extract the 9-dimensional physical features (Kurtosis, RMS, fault

frequencies). Training XGBoost models to establish our highly accurate, explainable baseline.

Week 5: Phase 1 (End-to-End DL & Comparison) Building and training the 1D-CNN to ingest the raw vibration waves. Comparing its accuracy and speed against the classical models developed in Week 4 to finalize the "domain knowledge vs. end-to-end learning" showdown.

Week 6: Phase 2 (Time to Vision & Computer Vision) Implementing Gramian Angular Fields (GAFs) to convert the 1D vibration time-series data into 2D visual heatmaps. Integrating pre-trained vision models (like ResNet) and fine-tuning them on these new images to test visual defect detection.

Week 7: Phase 2 (Generative Modeling) Learning the basics of generative AI and building the anomaly detection pipeline using Normalizing Flows. Training this unsupervised model strictly on "Healthy" machine data so it learns the baseline probability distribution of normal operations.

Week 8: Thresholding and Cross-Validation Establishing the statistical thresholds so the system can automatically flag failing machines as "anomalies." Running the ultimate real-world test by validating the completed anomaly detection system against the unseen secondary MFPT dataset.

Week 9: Refinement and Documentation Compiling the final evaluation metrics (Accuracy, F1-Score, Generalization Penalty). Cleaning up the code, adding detailed comments to the interactive Colab notebooks, and documenting the project's final findings for presentation.

References:

<https://medium.com/@RashmiW/vibration-analysis-of-bearings-for-early-fault-detection-a-matlab-case-study-1e2ff244c78e>

<https://www.mdpi.com/2410-390X/9/4/33>

<https://otexts.com/fpp3/seasonal-plots.html>

<https://www.youtube.com/watch?v=spUNpyF58BY>

https://pyts.readthedocs.io/en/stable/auto_examples/image/plot_dataset_gaf.html#sphx-glr-auto-examples-image-plot-dataset-gaf-py

<https://www.youtube.com/watch?v=ny1iZ5A8ilA>

Papers to support approach used in phase 1 :

<https://link.springer.com/article/10.1007/s42417-024-01405-0>

<https://icj-e.org/index.php/ojs/article/view/374/362>

Task Link:

https://drive.google.com/drive/folders/1V1GefcdZLKEEg5C_pqsnXVqeOL66a3E5?usp=drive_link